
CS204: Algorithms & Data Structures Lab

Week # 03 (1 Questions, 100 Marks)

Lab session: AL1

Held on: 12-Aug-2024 (Mon)

Lab Timings: 14:00 to 17:00 Hours Pages: 4

Submission time: 16:45 Hrs, 05-Aug-2024

Instructors Dr. V. Vijaya Saradhi

Head TAs Adithya K Moorthy & Laxita Agrawal

Department of CSE, IIT Guwahati

Useful and important **hints**:

1. For finding square root of a given number, call the in-built function `sqrt(.)` to achieve this.
2. Include `#include<math.h>`
3. Compile the program using `g++ 220101999-1.cc -lm`

Important note:

File naming Answer all the tasks of the question in a file named after your roll number. That is if your roll number is: 220101999 then first question file name should be 220101999-1.c

Question 1: (100 points)

Perform the following tasks. These tasks collectively lead to finding gradient at every element of the given image using two kernels. In other words, given three input matrices, perform specified computation to obtain a new matrix.

Task 01 (24 marks) Variable declaration for holding input data and reading input data:

1. (1 mark) Declare a structure named `matrix` having the following members
 - (a) `gx` of type `float`
 - (b) `gy` of type `float`
 - (c) `G` of type `float` and
 - (d) `value` of type `int`.
2. (1 mark) Declare a structure named `meta_data` having the following members
 - (a) `image_rows` of type `int`
 - (b) `image_columns` of type `int`
 - (c) `kx_rows` of type `int`
 - (d) `kx_columns` of type `int`
 - (e) `ky_rows` of type `int`
 - (f) `ky_columns` of type `int`
3. (1 mark) Declare a pointer to a pointer to the `matrix` structure namely k_x

4. (1 mark) Declare a pointer to a pointer to the `matrix` structure namely `ky`
5. (1 mark) Declare a pointer to a pointer to the `matrix` structure namely `image`
6. (1 mark) Read
 - (a) The number of rows into the variable `kx_rows` of `meta_data` from the input file `kx.txt`
 - (b) The number of columns into the variable `kx_columns` of `meta_data` from the input file `kx.txt`
7. (2 marks) Dynamically allocate memory for the variable `kx` of size `kx_rows` $\times$ `kx_columns`
8. (3 marks) Read the data from the input file `kx.txt` into into the variable `kx`.
9. (1 mark) Read
 - (a) The number of rows into the variable `ky_rows` of `meta_data` from the input file `ky.txt`
 - (b) The number of columns into the variable `ky_columns` of `meta_data` from the input file `ky.txt`
10. (2 marks) Dynamically allocate memory for the variable `ky` of size `ky_rows` $\times$ `ky_columns`
11. (3 marks) Read the data from the input file `ky.txt` into into the variable `ky`.
12. (1 mark) Read
 - (a) The number of rows into the variable `image_rows` of `meta_data` from the input file `image.txt`
 - (b) The number of columns into the variable `image_columns` of `meta_data` from the input file `image.txt`
13. (2 marks) Dynamically allocate memory for the variable `image` of size `image_rows` $\times$ `image_columns`
14. (4 marks) Read the data from the input file `image.txt` into into the variable `image`.

Task 02 - Study Image Gradient Computation Study how the image gradient computation works given an input image and two kernel matrices K_x and K_y from the slides `gradient-computation.pdf`.

Task 03 (10 Marks) Write three functions as described below

1. (4 marks) Write a function `compute_gx` whose specification is as follows:
 - (a) **Input arguments** this function takes the following inputs
 - i. A pointer-to-pointer to structure `matrix` namely, `kx`
 - ii. A pointer-to-pointer to structure `matrix` namely, `image`
 - iii. An `int` (stands for location i of image)
 - iv. An `int` (stands for location j of image)
 - v. The `meta_data` structure
 - (b) **return value** `None`.

- (c) **computation** This function should compute $g_x(i, j)$ as described in the slides and store it in **image**.
- 2. (4 marks) Write a function **compute_gy** whose specification is as follows:
 - (a) **Input arguments** this function takes the following inputs
 - i. A pointer-to-pointer to structure **matrix** namely, k_y
 - ii. A pointer-to-pointer to structure **matrix** namely, **image**
 - iii. An **int** (stands for location i of image)
 - iv. An **int** (stands for location j of image)
 - v. The **meta_data** structure
 - (b) **return value** None.
 - (c) **computation** This function should compute $g_y(i, j)$ as described in the slides and store it in **image**.
- 3. (2 marks) Write a function **compute_g** whose specification is as follows:
 - (a) **Input arguments** this function takes the following inputs
 - i. A pointer-to-pointer to structure **image** namely, **image**
 - ii. An **int** (stands for location i of image)
 - iii. An **int** (stands for location j of image)
 - (b) **return value** None.
 - (c) **computation** This function should compute $G(i, j)$ as described in the slides and store it in **image**.

Task 04 (10 marks) Compute $g_x(i, j)$, $g_y(i, j)$, and, $G(i, j)$ for all rows and columns of the **image** given k_x, k_y and **image**, i and j using the function developed in task 03.

Task 05 (5 marks) Write the output of the gradient in an output file **output-1.txt**. The format of the output file is:

```
( 1,  1): g_x: -442.00, g_y: -442.00, G: 625.08
( 2,  2): g_x: 241.00, g_y: 81.00, G: 254.25
( 3,  3): g_x: 360.00, g_y: 90.00, G: 371.08
...
```

Task 06 (10 marks) Lambda functions. Create a copy of **image** into **lambda_image**

- 1. (4 marks) Write a lambda function, **lambda_compute_gx**, whose specification is as follows: :
 - (a) **Input arguments** This function take the following arguments
 - i. capture list: pass a reference to k_x
 - ii. capture list: pass a reference to **lambda_image** of type pointer to pointer to structure **matrix**
 - iii. parameter 01 - **int i**
 - iv. parameter 02 - **int j**
 - (b) **return value** None

- (c) **computation** This function should compute $g_x(i, j)$
- 2. (4 marks) Write a lambda function, `lambda_compute_gy`, whose specification is as follows:
 - (a) **Input arguments** This function take the following arguments
 - i. capture list: pass a reference to k_y
 - ii. capture list: pass a reference to `lambda_image` of type pointer to pointer to structure `matrix`
 - iii. parameter 01 - `int i`
 - iv. parameter 02 - `int j`
 - (b) **return value** a float value
 - (c) **computation** This function should compute $g_y(i, j)$
- 3. (2 marks) Write a lambda function whose specification is as follows:
 - (a) **Input arguments** This function take the following arguments
 - i. parameter 01: g_x result obtained from `lambda_compute_gx`
 - ii. parameter 02: g_y result obtained from `lambda_compute_gy`
 - (b) **return value** a float value
 - (c) **computation** This function should compute $G(i, j)$

Task 07 (10 marks) For every pixel of the `lambda_image`, compute $G(i, j)$ using the lambda functions developed in Task 06.

Task 08 (5 marks) Repeat task 05 and write the output in a separate file named `output-2.txt`.

Task 09 (16 marks) function pointers.

1. (2 marks) Declare a function pointer which takes identical arguments as described for the `compute_gx` specified in task 03.
2. (2 marks) Declare a function pointer which takes identical arguments as described for the `compute_gy` specified in task 03.
3. (2 marks) Write a function pointer which takes identical arguments as described for the `compute_g` specified in task 03.
4. (5 marks) Use the above three function pointers to compute gradient of the image at every i, j that is $g_x(i, j)$, $g_y(i, j)$ and $G(i, j)$.
5. (5 marks) Repeat task 05 and write the output in a separate file `output-3.txt`.